ՀԱՅԱՍՏԱՆԻ ՀԱՆՐԱՊԵՏՈՒԹՅԱՆ ԿՐԹՈՒԹՅԱՆ,
ԳԻՏՈՒԹՅՈՒՆ, ՄՇԱԿՈՒՅԹԻ և ՍՊՈՐՏԻ ՆԱԽԱՐԱՐՈՒԹՅՈՒՆ
ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ

ՆԱԽԱԱՎԱՐՏԱԿԱՆ ՊՐԱԿՏԻԿԱ

ՀԱՇՎԵԲԱՑԱՏՐԱԳԻՐ

Թեմա՝ Ընկերության ծառայությունների կառավարման հարթակի մշակում

Ուսանող՝ Վարդանյան Թինա

Խումբ՝ ՏՏ219

Ղեկավար՝ Գ. Տոմեյան

Երևան 2026

Բովանդակություն

Ներածություն 2
1. Օգտագործված գործիքամիջոցների նկարագրություն 3
1.1. Frontend տեխնոլոգիաներ 3
1.2. Backend տեխնոլոգիաներ 7
1.3. Տվյալների բազա և պահպանում 10
2. Համակարգի ճարտարապետություն 11
3. Տվյալների հենքի նկարագրություն 13
4. Ալգորիթմների նկարագրություն 16
Եզրակացություն 18
Գրականության ցանկ 19

Ներածություն

Ժամանակակից թվային տնտեսությունում բիզնեսի կայունությունը պայմանավորված է ծառայությունների թվայնացման որակով, քանի որ ավանդական ապակենտրոնացված մոտեցումներն այլևս չեն բավարարում տեղեկատվական առատության պայմաններում գործող շուկայի պահանջներին: Թվայնացման պարադոքսը ստիպում է ընկերություններին կատարել ռիսկային ներդրումներ մարքեթինգում, մինչդեռ սպառողները դժվարանում են կողմնորոշվել առաջարկների բազմազանության մեջ: Այս խնդրի լուծման համար նախագծվել է B2B և B2C մոդելները համատեղող էլեկտրոնային հարթակ, որը հիմնված է բարձր տեխնոլոգիական ստեկի՝ React/Next.js frontend-ի, Node.js backend-ի և PostgreSQL/Redis տվյալների կառավարման համակարգերի վրա:

Աշխատանքի գիտական նորույթը կայանում է ժամանակակից web տեխնոլոգիաների սիներգիկ համակցման մեջ, որտեղ Next.js-ի Server-Side Rendering-ը լուծում է SEO օպտիմալացման հարցը, իսկ PostgreSQL-ի ACID հատկությունները և Redis-ի in-memory տեխնոլոգիան երաշխավորում են տվյալների ամբողջականությունն ու միլիվայրկյանային արձագանքը: Համակարգը մշակվել է Agile մեթոդաբանությամբ՝ ապահովելով մոդուլարություն և մասշտաբայնություն, ինչը թույլ է տալիս արդյունավետորեն թվայնացնել բարդ բիզնես գործընթացները և ստեղծել կենտրոնացված միջավայր ծառայությունների մատուցման ու սպառման համար:

1. Օգտագործված գործիքամիջոցների նկարագրություն

Համակարգի նախագծման և իրականացման ընթացքում ընտրվել է այնպիսի տեխնոլոգիական գործիքակազմ, որն ապահովում է համակարգի բարձր արագագործությունը, տվյալների հուսալիությունը, անվտանգությունը և մասշտաբայնությունը: Տեխնոլոգիաների ընտրությունը կատարվել է՝ հաշվի առնելով ժամանակակից best practices-ները, համայնքի աջակցությունը և production-ready լինելը:

1.1. Frontend տեխնոլոգիաներ

React 18.x - Հիմնական UI գրադարան

Որպես համակարգի հիմնական frontend գրադարան ընտրվել է React-ը, որը մշակվել է Meta (նախկին Facebook)-ի կողմից և այսօր համարվում է web development-ի ոսկե ստանդարտներից մեկը: React-ը JavaScript գրադարան է user interfaces-ների կառուցման համար, որը հիմնված է կոմպոնենտային ճարտարապետության վրա:

Ընտրությունը հիմնավորվում է հետևյալ առավելություններով՝

Կոմպոնենտային ճարտարապետություն: React-ի հիմնական փիլիսոփայությունը UI-ի բաժանումն է փոքր, վերաօգտագործելի կոմպոնենտների: Հարթակում յուրաքանչյուր UI տարր՝ ծառայության քարտ, փաթեթի կոնֆիգուրատոր, պատվերի ձև, ֆիլտրերի պանել՝ իրականացվում է որպես անկախ կոմպոնենտ: Սա թույլ է տալիս կրկնել կոմպոնենտները տարբեր էջերում՝ առանց կոդը կրկնելու, ինչը զգալիորեն նվազեցնում է սխալների հավանականությունը և հեշտացնում սպասարկումը:

Virtual DOM մեխանիզմ: React-ը օգտագործում է Virtual DOM (Document Object Model) հայեցակարգը, որը հիշողության մեջ պահում է իրական DOM-ի թեթև պատճենը: Երբ տվյալները փոխվում են, React-ը նախ թարմացնում է Virtual DOM-ը, համեմատում է դրան իրական DOM-ի հետ և միայն անհրաժեշտ փոփոխությունները կատարում է իրական DOM-ում: Սա զգալիորեն բարելավում է արտադրողականությունը՝ հատկապես դինամիկ ինտերֆեյսների դեպքում, ինչպիսին է ծառայությունների կատալոգը, որտեղ հաճախ են փոխվում ֆիլտրերը և դասավորման պարամետրերը:

Hooks API և ժամանակակից state management: React 16.8-ից սկսած ներդրվել է Hooks API-ն, որը հնարավորություն է տալիս functional components-ներում օգտագործել state և lifecycle հնարավորություններ: Հարթակում լայնորեն օգտագործվում են useState (local state), useEffect

(side effects), useContext (global state), useMemo և useCallback (performance optimization) hooks-ները: Բացի այդ, ստեղծվել են custom hooks-ներ՝ useAuth (authentication), useCart (shopping cart), useDebounce (search optimization) և այլն:

Հարուստ էկոհամակարգ և համայնք: React-ն ունի հսկայական օգտատերերի և developers-ների համայնք, ինչը նշանակում է, որ հասանելի են հազարավոր պատրաստի գրադարանների, կոմպոնենտների և լուծումների: Ցանկացած խնդրի համար գտնվում է ready-made լուծում կամ օգնություն community-ից: Բացի այդ, React-ը ակտիվորեն զարգացվում է և թարմացվում է Meta-ի և open-source community-ի կողմից:

Next.js 14.x - Production-Ready React Framework

Next.js-ը ընտրվել է որպես React-ի հիման վրա կառուցված full-stack framework: Այն ստեղծվել է Vercel ընկերության կողմից և համարվում է React applications-ների համար ամենահարմար production framework-ը:

Server-Side Rendering (SSR) և Static Site Generation (SSG): Next.js-ի ամենակարևոր առավելությունը hybrid rendering-ն է: Հարթակի հանրային էջերը (ծառայությունների կատալոգ, ընկերությունների պրոֆիլներ, ծառայությունների մանրամասն էջեր) ռենդերվում են սերվերում՝ Server-Side Rendering մեթոդով: Սա ապահովում է.

- Արագ առաջին բեռնում (First Contentful Paint)
- Լավ SEO՝ որոնողական համակարգերը տեսնում են լրիվ HTML content
- Social media previews՝ Open Graph և Twitter Cards համար
- Accessibility՝ content հասանելի է screen readers-ների համար

Ստատիկ էջերը (landing, about, help, terms) գեներացվում են build time-ում՝ Static Site Generation մեթոդով, ինչը ապահովում է lightning-fast բեռնում և նվազեցնում server load-ը:

File-based Routing համակարգ: Next.js-ում routing-ը ավտոմատ կերպով գեներացվում է ֆայլերի կառուցվածքից: Օրինակ՝ pages/services/[id].tsx ֆայլը ավտոմատ կերպով ստեղծում է /services/:id route: Սա զգալիորեն պարզեցնում է navigation-ի կառավարումը և հեշտացնում նոր էջերի ավելացումը:

API Routes և Backend Integration: Next.js-ը թույլ է տալիս ստեղծել API endpoints-ներ pages/api դիրեկտորիայում: Սա հարմար է serverless functions-ների համար, ինչպիսիք են

authentication callbacks, webhooks handling, image optimization և այլն: Հարթակում API routes-ները օգտագործվում են.

- Authentication flow-ների համար
- File uploads-ների համար
- Third-party API proxying-ի համար
- Server-side form validation-ի համար

Image Optimization և Performance: Next.js-ի Image կոմպոնենտը ավտոմատ կերպով օպտիմալացնում է պատկերները.

- Lazy loading՝ պատկերները բեռնվում են երբ մտնում են viewport
- Responsive images՝ ճիշտ չափի պատկերը ճիշտ device-ի համար
- Modern formats՝ WebP և AVIF format-ներ՝ browser support-ի դեպքում
- Blur placeholder՝ պատկերների բեռնման ժամանակ

Incremental Static Regeneration (ISR): Սա թույլ է տալիս թարմացնել ստատիկ էջերը առանց ամբողջ site-ը rebuild անելու: Օրինակ՝ ծառայության էջը կարող է ռեգեներացվել յուրաքանչյուր 60 վայրկյանը մեկ՝ ապահովելով թարմ content առանց SSR-ի performance penalty-ի:

TypeScript 5.x - Type Safety և Developer Experience

TypeScript-ը JavaScript-ի superset է, որն ավելացնում է ստատիկ տիպիզացիա: Մշակվել է Microsoft-ի կողմից և դարձել է web development-ի դե ֆակտո ստանդարտ:

Տիպերի անվտանգություն և սխալների կանխում: TypeScript-ի կոմպիլյատորը ստուգում է տիպերի համապատասխանությունը compile time-ում՝ բացահայտելով սխալները նախքան կոդը browser-ում կատարվի: Սա հատկապես կարևոր է բարդ state management-ի, API responses-ների և form validation-ի դեպքում:

IntelliSense և ավտո-լրացում: Modern IDE-ները (VS Code, WebStorm) լիովին օգտագործում են TypeScript-ի տիպերը՝ տրամադրելով.

- Intelligent code completion
- Parameter hints
- Quick documentation
- Automatic imports

- Refactoring tools

Interface-ներ և Type Aliases: TypeScript-ը թույլ է տալիս սահմանել տվյալների կառուցվածքների պայմանագրեր: Հարթակում սահմանված են տասնյակ interfaces և types՝ User, Company, Service, Package, Order, OrderItem, Review և այլն: Սրանք ապահովում են type safety ամբողջ application-ում:

Generic Types և Advanced Types: TypeScript-ի advanced type system-ը թույլ է տալիս ստեղծել reusable, type-safe utilities: Օգտագործվում են Utility Types՝ Partial, Pick, Omit, Record, mapped types և conditional types:

Tailwind CSS 3.x - Utility-First Styling

Tailwind CSS-ը utility-first CSS framework է, որն ապահովում է low-level utility classes:

Utility Classes և Rapid Development: Tailwind-ը տրամադրում է հարյուրավոր utility classes՝ margin, padding, colors, typography, flexbox, grid և այլն: Սա թույլ է տալիս արագ ստիլիզացնել կոմպոնենտները՝ առանց custom CSS գրելու: Օրինակ՝ ծառայության քարտը ստիլիզացվում է classes-ներով՝ flex, rounded-lg, shadow-md, hover:shadow-xl, transition-all և այլն:

Responsive Design առանց Media Queries: Tailwind-ը տրամադրում է responsive prefixes՝ sm:, md:, lg:, xl:, 2xl:: Սա թույլ է տալիս հեշտությամբ ստեղծել responsive layouts: Հարթակը լիովին responsive է և օպտիմալացված է բոլոր screen sizes-ների համար:

Customization և Theming: tailwind.config.js ֆայլում սահմանված են հարթակի բոլոր design tokens-ները՝ colors (primary, secondary, accent), spacing scale, typography scale, border radius values և այլն: Սա ապահովում է consistent design language ամբողջ application-ում:

JIT (Just-In-Time) Compiler և Production Optimization: Tailwind 3-ը օգտագործում է JIT compiler, որը.

- Գեներացնում է միայն օգտագործված classes-ները
- Թույլ է տալիս arbitrary values՝ w-[137px], bg-[#1c1c1c]
- Արագացնում է build time-ը
- Նվազեցնում է CSS bundle size-ը (հարթակում՝ ~10KB gzipped)

Shadcn/UI և Radix UI - Accessible Components

Shadcn/UI-ն collection է պատրաստի, accessible React կոմպոնենտների, որոնք կառուցված են Radix UI primitives-ների վրա:

Accessibility-First Approach: Բոլոր կոմպոնենտները համապատասխանում են WCAG 2.1 Level AA ստանդարտներին և ապահովում են.

- Keyboard navigation՝ Tab, Enter, Space, Arrow keys
- Screen reader support՝ ARIA labels և roles
- Focus management՝ visible focus indicators
- Color contrast՝ minimum 4.5:1 ratio

Unstyled Primitives և Customization: Radix UI-ն տրամադրում է unstyled, headless components: Սրանք ապահովում են functionality և accessibility՝ թողնելով styling-ը developer-ի վրա: Shadcn/UI-ն վերցնում է Radix primitives-ները և ավելացնում Tailwind styling:

Օգտագործվող կոմպոնենտները.

- Dialog՝ modal windows և confirmations
- Dropdown Menu՝ user menus և actions
- Select՝ searchable dropdowns
- Tooltip՝ contextual help
- Toast՝ notifications
- Accordion՝ FAQ և collapsible content
- Tabs՝ content organization
- Form elements՝ Input, Textarea, Checkbox, Radio

1.2. Backend տեխնոլոգիաներ

Node.js-ը JavaScript runtime է, որը կառուցված է Chrome-ի V8 JavaScript engine-ի վրա:

Event-Driven, Non-Blocking I/O Architecture: Node.js-ն օգտագործում է single-threaded event loop մոդել՝ asynchronous, non-blocking operations-ների համար: Սա նշանակում է, որ մեկ thread-ը կարող է handle անել հազարավոր concurrent connections: Օրինակ՝ երբ server-ը սպասում է database query-ի պատասխանին, այն չի block անում և կարող է մշակել այլ requests:

High Throughput և Scalability: Node.js-ը իդեալական է I/O-intensive applications-ների համար՝ ինչպիսիք են web APIs, real-time applications, streaming services: Benchmark-ները ցույց են տալիս, որ Node.js-ը կարող է handle անել 10,000+ concurrent connections single instance-ում:

NPM Ecosystem: NPM (Node Package Manager)-ը ամենամեծ software registry-ն է աշխարհում՝ 2+ million packages: Հարթակում օգտագործվում է 50+ npm packages՝ authentication-ից մինչև image processing:

JavaScript Everywhere: Նույն լեզուն frontend-ում և backend-ում հեշտացնում է.

- Code sharing՝ validation schemas, utility functions, type definitions
- Developer experience՝ ավելի քիչ context switching
- Full-stack development՝ մեկ developer կարող է աշխատել ամբողջ stack-ի վրա

Express.js 4.x - Web Application Framework

Express-ը minimalist, unopinionated web framework է Node.js-ի համար:

Middleware Architecture: Express-ի հիմնական concept-ը middleware functions-ն են: Middleware-ը function է, որը ունի access request և response objects-ներին: Middleware chain-ը թույլ է տալիս իրականացնել.

- Request logging
- Authentication և authorization
- Body parsing (JSON, URL-encoded)
- CORS handling
- Error handling
- Rate limiting
- Request validation

Routing System: Express Router-ը թույլ է տալիս կազմակերպել routes-ները մոդուլային կերպով: Հարթակում routes-ները բաժանված են.

- /api/auth՝ authentication endpoints
- /api/users՝ user management
- /api/companies՝ company operations
- /api/services՝ service catalog
- /api/packages՝ package management

- /api/orders՝ order processing
- /api/reviews՝ review system

Performance և Minimalism: Express-ը չի պարտադրում որոշակի structure կամ dependencies: Սա թույլ է տալիս ստեղծել lightweight, performant applications: Express-ի overhead-ը minimal է՝ միայն ~881KB dependencies:

Prisma ORM - Next-Generation Database Toolkit

Prisma-ն modern ORM է, որը վերափոխում է database workflow-ը:

Prisma Schema - Declarative Data Modeling: Prisma Schema-ն single source of truth է database-ի համար: Այն սահմանում է models, relations, indexes և այլն պարզ, declarative syntax-ով: Schema-ն գրված է Prisma Schema Language-ով և պարունակում է բոլոր database models-ները:

Type-Safe Database Client: Prisma Client-ը auto-generated է և լիովին type-safe: Յուրաքանչյուր database query-ն ունի ճշգրիտ TypeScript types: Սա կանխում է.

- Runtime errors՝ invalid queries-ը բռնվում են compile time-ում
- SQL injection՝ բոլոր queries-ը parameterized են
- Type mismatches՝ database types-ները match են TypeScript types-ներին

Prisma Migrate - Version Control for Database: Prisma Migrate-ը database schema migrations-ների tool է: Այն.

- Գեներացնում է SQL migration files-ներ schema changes-ից
- Պահպանում է migrations history
- Թույլ է տալիս rollback անել
- Աջակցում է team collaboration-ի

Query Performance և Optimization: Prisma-ն օպտիմալացնում է queries-ները.

- Intelligent query planning
- N+1 query problem-ի prevention
- Dataloader pattern implementation

- Connection pooling

Prisma Studio - Database GUI: Prisma Studio-ն visual editor է database-ի համար: Թույլ է տալիս.

- View և edit data
- Run queries
- Explore relations
- Debug data issues

1.3. Տվյալների բազա և պահպանում

PostgreSQL 15.x – Ռելյացիոն տվյալների բազա

PostgreSQL-ն ընտրվել է որպես հիմնական պահոց՝ շնորհիվ իր հուսալիության և բարդ տվյալների հետ աշխատելու հնարավորության.

- **ACID համապատասխանություն:** Երաշխավորում է տրանզակցիաների ամբողջականությունը և տվյալների վավերականությունը (Consistency):
- **Տվյալների առաջադեմ տիպեր:** Աջակցում է JSONB (կիսակառուցվածքային տվյալներ), UUID, Arrays և Range types, ինչը թույլ է տալիս ճկուն մոդելավորել ծառայությունների գնային սանդղակները:
- **Full-Text Search:** Ներկառուցված tsvector և tsquery գործիքակազմը՝ GIN ինդեքսների կիրառմամբ, ապահովում է արագ և լեզվակիրառական որոնում հարթակում:
- **Մասշտաբավորում:** Աջակցում է Table Partitioning-ը և Materialized views-ը՝ մեծածավալ տվյալների պայմաններում բարձր արդյունավետություն պահպանելու համար:

Redis 7.x – In-Memory տվյալների պահոց

Redis-ն օգտագործվում է որպես օժանդակ շերտ՝ արագագործությունը բարելավելու և հիմնական բազայի բեռնվածությունը նվազեցնելու նպատակով.

- **Բարձր արդյունավետություն:** RAM-ում տվյալների պահպանման շնորհիվ ապահովում է <1մվ հապաղում (latency) և վայրկյանում ավելի քան 100,000 գործողություն:
- **Կիրառման ոլորտները:**
  - **Caching:** Հաճախակի հարցվող API պատասխանների քեշավորում:
  - **Session Management:** Օգտատերերի սեսիաների և JWT refresh տոկենների պահպանում:
  - **Rate Limiting:** Հարցումների քանակի սահմանափակում (Request counters):
  - **Ժամանակավոր տվյալներ:** OTP կոդերի և վերիֆիկացիոն տոկենների կառավարում:
- **Տվյալների պահպանելիություն:** Ի տարբերություն սովորական քեշավորման համակարգերի, Redis-ն աջակցում է RDB snapshot-ներ և AOF լոգավորում՝ տվյալների կայունությունն ապահովելու համար:

2. Համակարգի ճարտարապետություն

Հարթակի ծրագրային կառուցվածքը հիմնված է **բազմաշերտ ճարտարապետության** (N-Tier Architecture) և **Separation of Concerns (SoC)** սկզբունքների վրա: Սա ապահովում է պատասխանատվության գոտիների հստակ տարանջատում, մոդուլարություն և համակարգի դյուրին մասշտաբավորում:

Ներկայացման շերտ (Frontend)

Frontend-ը իրականացված է **React.js** և **Next.js** տեխնոլոգիաներով՝ համատեղելով SPA-ի ճկունությունը և Server-Side Rendering-ի (SSR) արդյունավետությունը: Ֆունկցիոնալությունը բաժանված է ըստ օգտատերերի դերերի.

- **Հանրային տիրույթ:** Ներառում է Landing Page-ը, ծառայությունների կատալոգը՝ բազմամակարդակ ֆիլտրացմամբ և Full-text search-ով, ինչպես նաև ընկերությունների հանրային պրոֆիլները:
- **Օգտատերերի դաշբորդներ:** Կլիենտների համար գործում է **Package Configurator**(ծառայությունների անհատական փաթեթավորման գործիք), պատվերների կառավարում և զամբյուղ: Ընկերությունների համար ներդրված է CRUD ինտերֆեյս՝ ծառայությունների և փաթեթների կառավարման, ինչպես նաև վաճառքների վերլուծության (Analytics Dashboard) համար:
- **Ադմինիստրատորի պանել:** Նախատեսված է օգտատերերի մոդերացիայի, բովանդակության վերահսկման և համակարգի ընդհանուր վիճակագրության մոնիթորինգի համար:

API Gateway և անվտանգության շերտ

Backend-ի մուտքի կետը **Express.js** հիմքով API Gateway-ն է, որը կատարում է հետևյալ կրիտիկական ֆունկցիաները.

- **JWT Authentication & RBAC:** Իրականացվում է երկշերտ տոկենային համակարգով (Access/Refresh tokens) և դերերի վրա հիմնված հասանելիության հսկողությամբ (CLIENT, COMPANY, ADMIN):
- **Request Validation:** Յուրաքանչյուր հարցում վալիդացվում է **Zod** գրադարանի միջոցով՝ նախքան բիզնես տրամաբանությանը փոխանցվելը:
- **Rate Limiting:** Redis-ի հիման վրա գործող սահմանափակումները պաշտպանում են համակարգը DDoS հարձակումներից և չարաշահումներից:

- **Error Handling & CORS:** Կենտրոնացված մեխանիզմը ստանդարտացնում է սխալների պատասխանները, իսկ CORS կարգավորումները սահմանափակում են հասանելիությունը միայն վստահելի տիրույթներից:

Կիրառական և տվյալների շերտ (Business Logic)

Բիզնես տրամաբանությունը կազմակերպված է չորս ենթաշերտերով.

1. **Handler/Controller Layer:** Ընդունում է HTTP հարցումները և վերադարձնում ձևավորված պատասխանները:
2. **Service Layer:** Իրականացնում է հիմնական բիզնես կանոնները, կառավարում է քեշավորումը և արտաքին ինտեգրումները:
3. **Repository Layer:** Մեկուսացնում է տվյալների բազայի հետ աշխատանքը՝ օգտագործելով **Prisma Client**:
4. **Models Layer:** Սահմանում է տվյալների սխեմաները և հարաբերությունները (Relations) Prisma-ի միջոցով:

3. Տվյալների հենքի նկարագրություն

Տվյալների հենքը համակարգի հիմքն է և պատասխանատու է տեղեկատվության հուսալի պահպանման, արագ հասանելիության և ամբողջականության ապահովման համար: Տվյալների մոդելը նախագծված է ռելյացիոն մոդելի վրա՝ օգտագործելով PostgreSQL RDBMS-ը և հետևելով database normalization սկզբունքներին:

Տվյալների բազայի սխեման նորմալացված է մինչև Third Normal Form (3NF): Սա նշանակում է, որ.

- Յուրաքանչյուր աղյուսակ ներկայացնում է մեկ entity
- Չկա partial dependency primary key-ի non-key attributes-ների վրա
- Չկա transitive dependency non-key attributes-ների միջև
- Redundancy-ն minimum է
- Data anomalies (insertion, update, deletion) են կանխվում

Data integrity-ն ապահովվում է.

- Primary Keys՝ unique identifiers (UUID format)
- Foreign Keys՝ referential integrity
- Unique Constraints՝ prevent duplicates
- Check Constraints՝ domain validation
- Not Null Constraints՝ mandatory fields

Օգտատերերի մոդելներ

users աղյուսակ - Հիմնական authentication և authorization տվյալներ: Պարունակում է email (unique), password_hash (bcrypt), role (enum), verification status, activity status: Timestamps-ները track են անում ստեղծման և թարմացման ժամանակները:

user_profiles աղյուսակ - Extended user information: One-to-one relation users-ի հետ: Պարունակում է անձնական տվյալներ (first_name, last_name, phone), avatar_url, bio, location: preferences դաշտը (JSONB type) պահպանում է ճկուն settings (language, currency, notifications, privacy):

Ընկերությունների մոդելներ

companies աղյուսակ - Ընկերությունների հիմնական տվյալներ: One-to-one relation users-ի հետ (մեկ user՝ մեկ company): Պարունակում է company_name, legal_name, tax_id, description, logo_url, website: Verification fields (is_verified, verified_at) track են անում verification status-ը: Denormalized fields՝ average_rating, total_reviews (performance optimization):

company_addresses աղյուսակ - Ընկերությունների հասցեներ: One-to-many relation companies-ի հետ: Address type (MAIN, BRANCH, BILLING): Geographic data՝ country, city, street, postal_code, latitude, longitude: isPrimary flag main address-ի համար:

Ծառայությունների մոդելներ

categories աղյուսակ - Hierarchical category system: Self-referential relation (parent_id → categories.id): Fields՝ name, slug (URL-friendly), description, icon, order: Supports unlimited nesting depth:

services աղյուսակ - Core entity: Many-to-one relations՝ companies, categories: Fields՝ title, slug, description, short_description: Pricing՝ price, price_type (FIXED/HOURLY/DAILY/CUSTOM), duration: Metadata՝ is_active, views_count: Full-text search index՝ title և description fields-ների վրա:

service_images աղյուսակ - Service image gallery: One-to-many relation services-ի հետ: Fields՝ image_url, alt_text, order, is_primary: Supports multiple images per service:

Փաթեթների մոդելներ

packages աղյուսակ - Service bundles: Many-to-one relation companies-ի հետ: Fields՝ name, slug, description: Pricing՝ total_price, discount_percent, final_price: final_price-ը calculated field է (total_price - discount):

package_services աղյուսակ - Join table: Many-to-many relation packages և services-ի միջև: Fields՝ quantity, price_override: price_override-ը override անում է service-ի default price-ը package context-ում: Unique constraint՝ (package_id, service_id):

Պատվերների մոդելներ

orders աղյուսակ - Customer orders: Many-to-one relation users-ի հետ: order_number (unique, human-readable): Pricing՝ total_amount: Status tracking՝ status

(PENDING/CONFIRMED/IN_PROGRESS/COMPLETED/CANCELLED), payment_status (PENDING/PAID/FAILED/REFUNDED), payment_method (ONLINE/INVOICE/CASH): notes field additional instructions-ների համար:

order_items աղյուսակ - Order line items: One-to-many relation orders-ի հետ: Many-to-one relation companies-ի հետ: Polymorphic-like structure՝ item_type (SERVICE/PACKAGE), item_id (service_id or package_id): Snapshot fields՝ item_name, unit_price, total_price (պահպանում են values order creation-ի ժամանակ): item_status (PENDING/CONFIRMED/IN_PROGRESS/DELIVERED/CANCELLED) individual tracking-ի համար:

Գնահատականների մոդել

reviews աղյուսակ - User reviews: Many-to-one relations՝ companies, users: Optional relation orders-ի հետ: rating (INTEGER, 1-5 range): title և comment fields: Moderation fields՝ is_verified (verified purchase), is_approved (admin approval): Unique constraint՝ (order_id, user_id, company_id) prevents duplicate reviews:

Պորտֆոլիոյի մոդել

portfolio_items աղյուսակ - Company portfolio: One-to-many relation companies-ի հետ: Fields՝ title, description, image_url, project_url, client_name, completion_date: tags (Array type) categorization-ի համար: order field manual ordering-ի համար: is_featured flag featured items-ի համար:

Ծանուցումների մոդել

notifications աղյուսակ - User notifications: One-to-many relation users-ի հետ: type field notification category-ի համար: title և message fields: data field (JSON type) additional metadata-ի համար: is_read flag tracking համար:

4. Ալգորիթմների նկարագրություն

Հարթակի արդյունավետությունը և օգտատերերի փորձառությունը (UX) հիմնված են հետևյալ առանցքային ալգորիթմների վրա.

Որոնման և ֆիլտրացման համակարգ

Իրականացվել է բազմամակարդակ որոնման մեխանիզմ՝ հիմնված **PostgreSQL Full-Text Search**-ի վրա.

- **Լեզվական օպտիմալացում:** Օգտագործվում են հայերենի համար հարմարեցված tsvector և tsquery տիպերը՝ հաշվի առնելով բառարմատները (stemming):
- **Ranking և Sorting:** Արդյունքները դասակարգվում են ըստ ռելևանտության (term frequency/position), գնի, վարկանիշի և թարմության:
- **Performance:** Կիրառվում է **Debounced search** (300մվ) և Redis քեշավորում՝ հանրամատչելի հարցումների համար:

Փաթեթավորման (Package Builder) ալգորիթմ

Թույլ է տալիս օգտատերերին ստեղծել անհատականացված ծառայությունների փաթեթներ.

- **Դինամիկ գնագոյացում:** Համակարգը ավտոմատ հաշվարկում է զեղչերը՝ ըստ ծառայությունների քանակի (օրինակ՝ 5+ ծառայություն = 15% զեղչ):

Smart Suggestions: Համադրում է **Collaborative filtering** (այլ օգտատերերի ընտրություն) և **Content-based** (նույն կատեգորիայի ծառայություններ) մոտեցումները՝ լրացուցիչ առաջարկներ **գեներացնելու համար:**

Պատվերների և Վճարումների մշակում

Պատվերի գրանցումն իրականացվում է **Atomic Transaction** սկզբունքով, ինչը երաշխավորում է, որ գործընթացի որևէ փուլում սխալի դեպքում տվյալները կվերադառնան նախնական վիճակին (rollback):

- **State Machine:** Պատվերն անցնում է հստակ փուլերով (PENDING → CONFIRMED → COMPLETED), որոնցից յուրաքանչյուրը տրիգեր է հանդիսանում ծանուցումների և հաշիվների գեներացման համար:

- **Gateway Integration:** Ինտեգրված է Stripe/PayPal վճարային համակարգերի հետ՝ webhook-ների միջոցով վճարման կարգավիճակի ավտոմատ թարմացմամբ:

Վարկանիշի հաշվարկ (Weighted Average)

Պարզ միջինի փոխարեն կիրառվում է **կշռված միջինի** մեթոդը.

- **Ժամանակային գործոն:** Վերջին 6 ամսվա կարծիքներն ունեն ավելի մեծ կշիռ (1.5), քան հին կարծիքները (0.5):
- **Վստահելիություն:** Հաստատված գնումներով (Verified purchase) կարծիքներն ավելի մեծ ազդեցություն ունեն ընկերության ընդհանուր վարկանիշի վրա:

Քեշավորման ռազմավարություն (Cache-Aside)

Արագագործության համար կիրառվում է Redis **Cache-Aside** մոդելը.

- **TTL և Invalidation:** Յուրաքանչյուր տվյալ ունի իր կյանքի տևողությունը (Service listings՝ 5ր, Sessions՝ 7օր): Տվյալների թարմացման դեպքում (Event-based) քեշն ավտոմատ մաքրվում է:
- **Cache Warming:** Հանրաճանաչ ծառայությունները և գլխավոր էջի տվյալները նախապես բեռնվում են քեշում՝ նվազագույն latency ապահովելու համար:

Եզրակացություն

Ընկերությունների ծառայությունների մատուցման հարթակի մշակման ընթացքում իրականացվել է համապարփակ տեխնիկական լուծում, որը համադրում է React/Next.js frontend-ի արագագործությունը Node.js/Express backend-ի բարձր արդյունավետության հետ: Համակարգի հիմքում ընկած է PostgreSQL տվյալների բազան՝ Prisma ORM-ի միջոցով ապահովված տիպայնացված կոդով և տվյալների ամբողջականությամբ, իսկ Redis քեշավորման շերտը երաշխավորում է արձագանքման նվազագույն հապաղումներ: Բազմաշերտ ճարտարապետությունը և separation of concerns սկզբունքը թույլ են տվել ստեղծել թեստավորվող և հեշտ սպասարկվող միջավայր, որտեղ անվտանգությունն ապահովված է JWT վավերացման, դերերի վրա հիմնված հասանելիության (RBAC) և տվյալների էնկրիպտացիայի միջոցով: Docker կոնտեյներացման և CI/CD գործընթացների ներդրումը ավտոմատացրել է տեղակայումը՝ ապահովելով միջավայրերի կայունությունը:

Իրականացված ալգորիթմները՝ սկսած տեքստային որոնումից մինչև փաթեթների կառուցում, օպտիմալացնում են օգտատերերի փորձառությունը, իսկ նախագծի մոդուլյարությունը հիմք է ստեղծում ապագայում ML առաջարկների և իրական ժամանակում գործող ֆունկցիաների ներդրման համար: Ամփոփելով՝ ստեղծված էլեկտրոնային հարթակը ներկայացնում է ժամանակակից, մասշտաբավորվող և արտադրական շահագործման պատրաստ լուծում B2B և B2C շուկաների համար:

Գրականության ցանկ

1. Banks A., Porcello E. Learning React: Modern Patterns for Developing React Apps. 2nd Edition. O'Reilly Media, 2020. 310 p.
2. Wilson J. Node.js 8 the Right Way: Practical, Server-Side JavaScript That Scales. Pragmatic Bookshelf, 2018. 350 p.
3. Obe R., Hsu L. PostgreSQL: Up and Running. 3rd Edition. O'Reilly Media, 2017. 310 p.
4. Carlson J. Redis in Action. Manning Publications, 2013. 320 p.
5. Richardson C. Microservices Patterns: With Examples in Java. Manning Publications, 2018. 520 p.
6. Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media, 2017. 616 p.
7. Newman S. Building Microservices: Designing Fine-Grained Systems. 2nd Edition. O'Reilly Media, 2021. 612 p.
8. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley Professional, 2002. 560 p.
9. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional, 1994. 416 p.
10. Hunt A., Thomas D. The Pragmatic Programmer: Your Journey to Mastery. 2nd Edition. Addison-Wesley Professional, 2019. 352 p.